

MLcreate: A Cloud-Native Machine Learning Pipeline

Abstract

MLcreate is a cloud-based machine learning platform that enables authenticated users to upload labeled CSV datasets, train predictive models, and run batch inference through a web application. The system uses an asynchronous cloud architecture built with Next.js, AWS SQS, Amazon S3, Amazon RDS for PostgreSQL, and containerized Python workers. The pipeline supports both classification and regression tasks, with preprocessing handled by PySpark and model training implemented through Spark XGBoost with local fallbacks. The project demonstrates how cloud services and open-source ML frameworks can be combined to build a practical end-to-end machine learning workflow.

1 Introduction

MLcreate is a cloud-based machine learning pipeline that allows authenticated users to upload a labeled CSV dataset, train a model, and run batch predictions through a web interface. The project combines a modern frontend with an asynchronous backend pipeline built around AWS services and containerized Python workers.

Based on the current implementation, MLcreate is best described as a cloud-native ML application rather than a pure research prototype. Its goal is to automate the full workflow of dataset ingestion, preprocessing, training, model storage, and prediction delivery while keeping the user-facing application responsive.

2 System Architecture

The system is divided into two main parts:

- A Next.js web application that handles user interaction, authentication, uploads, model browsing, and prediction requests.
- A set of Python consumer services, deployed as Docker containers on Amazon EC2, that execute the heavy ML pipeline asynchronously.

The overall workflow is:

Browser → Next.js API → AWS SQS → Dockerized consumers → S3 / Amazon RDS PostgreSQL → Browser

The implemented pipeline stages are:

1. **Upload:** the user uploads a CSV file and selects either classification or regression.
2. **Ingest:** the upload API stores the file in S3 and sends a message to the first SQS queue.
3. **Preprocess:** a preprocessing consumer reads the uploaded dataset, launches a PySpark job, and writes processed Parquet output.
4. **Train:** a training consumer loads the processed data and trains a model, preferring Spark/XGBoost and falling back to local models when necessary.
5. **Predict:** a prediction consumer loads the trained model, runs inference on a new CSV, writes the output CSV, and uploads the result to S3.

This design keeps the system asynchronous and decoupled. The web layer does not perform expensive ML work directly; it only triggers jobs and polls for status updates.

Table 1: Main cloud components used in MLcreate

Component	Purpose
Next.js + Clerk	frontend, authentication, API routes
AWS SQS	asynchronous messaging between pipeline stages
Amazon S3	storage for datasets, models, and prediction outputs
Amazon RDS PostgreSQL	job metadata, model records, and metrics
Dockerized Python consumers	ingestion, preprocessing, training, prediction
PySpark / XGBoost	preprocessing and model training

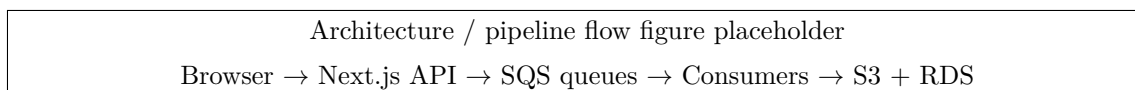


Figure 1: High-level MLcreate architecture. Replace this placeholder with a final system diagram if desired.

3 Technology Stack

3.1 Frontend

The frontend is implemented using Next.js 16, React 19, and TypeScript. Clerk is used for authentication and route protection.

Implemented frontend features include:

- user sign-in and sign-up
- model creation page
- CSV upload form
- task-type selection for classification or regression
- job status polling
- per-user model listing
- prediction upload and downloadable output

The upload route creates a unique job ID, uploads the dataset to S3, inserts a job record into PostgreSQL, and publishes an SQS message to start the pipeline.

3.2 Backend and Infrastructure

The backend relies on the following cloud and service components:

- AWS SQS for asynchronous messaging between pipeline stages
- AWS S3 for storing uploads, processed data, trained models, and prediction outputs
- PostgreSQL hosted on Amazon RDS for job and model metadata
- Amazon EC2 as the main compute host for the backend pipeline
- Docker Compose for orchestrating the containerized consumer services on EC2

Amazon EC2 played a central role in the deployed system. While SQS, S3, and RDS provided managed cloud services, EC2 provided the persistent compute environment where the long-running backend workers executed. The ingestion, preprocessing, training, and prediction services were packaged as Docker containers and run on EC2, allowing the system to continuously poll queues, process jobs, access shared configuration, and manage runtime dependencies in a controlled environment.

There are four main queue-driven consumer services:

- `ingest_consumer`
- `preprocess_consumer`
- `train_consumer`
- `predict_consumer`

3.3 Machine Learning Stack

The ML and data-processing stack includes:

- PySpark
- XGBoost
- scikit-learn
- pandas
- pyarrow
- joblib

4 Implemented Functionality

4.1 Queue-Driven Processing Pipeline

The backend is implemented as multiple long-running Python consumers, each polling one SQS queue and invoking the corresponding handler. This architecture provides several cloud-computing benefits:

- loose coupling between stages
- asynchronous execution
- better fault isolation
- easier future scaling by consumer type

Each stage communicates through queue messages and stored artifacts rather than direct service-to-service invocation.

4.2 Data Ingestion and Storage

Training CSV files are stored under keys such as:

- `uploads/<job_id>/<filename>`

Processed datasets are stored as Parquet files:

- `processed/<job_id>.parquet`

Models are stored as:

- `models/<job_id>.pkl`
- `models/<job_id>.pkl.spark`
- `models/<job_id>.pkl.meta`

Prediction outputs are stored as:

- `predictions_output/<predict_id>.csv`

The project also keeps local copies during development for compatibility with Docker-based execution.

4.3 Preprocessing with Spark

The preprocessing stage is implemented in a PySpark job. The current logic includes:

- reading CSV input with inferred schema
- dropping rows whose `label` field is null
- removing duplicate `id` values if present
- writing the cleaned output as Parquet

This is a simple but valid distributed-data-processing step that prepares the dataset for downstream model training.

4.4 Model Training

The training pipeline uses a layered fallback design.

The preferred path is Spark-based training using:

- `SparkXGBClassifier`
- `SparkXGBRegressor`

If Spark is unavailable or fails, the system falls back to:

- local XGBoost
- `LinearRegression` for regression
- majority-class and mean-value predictors as final fallbacks

This fallback strategy is one of the strongest aspects of the implementation because it keeps the pipeline operational across different runtime environments.

The training stage computes evaluation metrics as follows:

- classification: accuracy
- regression: MSE, RMSE, and R^2

Metadata is then recorded into PostgreSQL, and completed jobs are reflected in the `models` table.

4.5 Batch Prediction

The prediction pipeline accepts a new CSV for an existing model, loads either a Spark model or a joblib model, generates predictions, writes them to CSV, uploads the output to S3, and exposes a downloadable link to the user through the frontend.

This completes the full ML lifecycle:

- data upload
- preprocessing
- training
- model persistence
- inference
- result retrieval

5 Database Design

The PostgreSQL schema contains three main tables:

- `jobs`
- `models`
- `metrics`

The `jobs` table stores job identity, model name, task type, ownership, and status.

The `models` table stores completed model records and model paths.

The `metrics` table stores evaluation results.

In the deployed cloud setup, the PostgreSQL database was hosted on Amazon RDS, while a local PostgreSQL container was used for development and testing. This schema is lightweight but sufficient for the implemented workflow, and it supports per-user model isolation in the frontend.

Table 2: Database tables and their roles

Table	Role
<code>jobs</code>	tracks uploaded jobs, task types, user ownership, and status
<code>models</code>	stores completed model artifacts and model paths
<code>metrics</code>	stores evaluation scores such as accuracy, MSE, RMSE, and R^2

6 Cloud Computing Aspects

This project fits the cloud-computing theme because it applies several core cloud-system ideas to machine learning deployment:

- managed cloud services are used for messaging and storage
- the backend is decomposed into asynchronous workers
- compute-heavy processing is separated from the web frontend
- artifacts are externalized into object storage
- metadata is persisted in a shared database hosted on Amazon RDS
- Dockerized workers make deployment portable

In effect, MLcreate behaves like a small ML platform built from cloud services plus open-source ML frameworks.

7 Current Limitations

Current limitations:

- Spark is configured in `local[*]` mode, so training is not yet truly multi-node distributed.
- Kubernetes deployment is not finished; the `k8s` folder is mostly a placeholder.
- Some legacy Kafka-related remnants still remain in the repository, although the active pipeline uses SQS.
- Preprocessing is minimal and does not yet include robust encoding, imputation, or feature engineering.
- Prediction assumes that the uploaded inference CSV matches the training schema.
- The system appears designed around one main consumer host rather than horizontally scaled worker pools.

8 Conclusion

MLcreate successfully implements a working cloud-native machine learning workflow with a real web frontend, asynchronous queue-based backend, cloud object storage, and database-backed metadata tracking. The most substantial implementation work lies in orchestrating the end-to-end ML lifecycle across Next.js, SQS, S3, Spark, Amazon RDS PostgreSQL, and Python consumers.

Although the platform is not yet a fully distributed Kubernetes-based ML system, it already demonstrates a strong practical cloud-computing design: decoupled services, asynchronous job execution, cloud storage integration, managed relational metadata storage, and containerized ML processing. It is therefore a solid course project in the area of cloud-based machine learning platforms.